

Analysis of TorchSig `clock_drift` and `clock_jitter` Transforms

Conceptual review and implementation audit of the sampling-clock impairment models in the provided `dsp.py` and `functional.py`

August 27, 2026

1. Scope and Method

This report analyzes the implementations of the `clock_drift` and `clock_jitter` functional transforms and, since both are thin wrappers around a shared engine, the full supporting chain: `sampling_clock_impairments`, `partition_polyphase`, and the prototype filter design (`prototype_polyphase_filter`, `low_pass_iterative_design`, `estimate_filter_length`) in `dsp.py`. Findings come from static analysis plus empirical verification: the functions were extracted verbatim into a standalone harness and exercised with $N = 4096$ complex tones at the exact parameters the wrappers use (uprate = 5000, the 120 dB prototype), with seeded random number generators for reproducibility.

`functional.py` prefers a numba-accelerated variant imported from `dsp_numba.py`, which was not provided. All conclusions therefore apply to the pure NumPy reference path, which the numba path presumably mirrors; any fixes must be applied to both.

2. What the Code Actually Does

Both transforms wrap one arbitrary-phase polyphase interpolator. Conceptually the input is upsampled by $L = 5000$ through a lowpass prototype (iteratively designed to a 120 dB stopband; the design converges to 115,101 taps, giving 24 taps per branch), and output samples are produced by walking a position pointer along that fine grid. For `clock_drift`, the pointer step is $L(1 + \text{drift_ppm} \times 1e-6)$, a constant sampling-rate offset. For `clock_jitter`, the step is exactly L , but each output's read position is perturbed by an independent zero-mean Gaussian of standard deviation $L \times \text{jitter_ppm} \times 1e-6$ fine-grid units, deliberately non-accumulating. Both wrappers then force the output back to the input length by trimming (or zero-padding) symmetrically.

3. Conceptual Soundness

The two underlying models are legitimate, standard ways to represent the two limiting components of a non-ideal sampling clock.

3.1 Drift as a constant rate offset

Modeling “drift” as a constant fractional resampling of the band-limited signal, $t_k = k(1 + d)T$, is the textbook sampling-rate-offset model and a defensible simplification over a short capture, where an oscillator's frequency is effectively constant. Two caveats apply. First, terminology: what is implemented is a static clock frequency offset (skew), not drift; there is no time variation at all. The wrapper docstring compounds this by saying “Data with LO drift applied,” which is wrong twice, since this is a sampling-clock effect, not a local-oscillator effect, and it is an offset, not drift. Second, in a real receiver the ADC clock and the LO usually share a reference, so a d ppm sampling offset co-occurs with a d ppm carrier offset scaled by the carrier frequency; decoupling them into separate composable transforms is a reasonable design choice, but it should be documented, because applying `clock_drift` alone does not correspond to any single

physical oscillator error.

3.2 Jitter as independent Gaussian timing perturbation

Modeling jitter as independent Gaussian perturbation of the sampling instants of the band-limited reconstruction is exactly the standard model of ADC aperture jitter, and realizing it by interpolating the discrete signal at jittered instants is the correct surrogate. The explicit non-accumulation is correct for aperture jitter and is documented. The caveat is scope: real sampling-clock timing noise also has a colored, accumulating component (integrated oscillator phase noise, wander), and at communications timescales that component usually dominates the observable effect. White-only jitter plus constant-only offset spans the two extremes of clock misbehavior but omits the random-walk middle, so as a pair they are a partial model of non-ideal clock impairments, sound as far as they go.

3.3 Magnitude context

At the defaults, both impairments are physically minuscule. Ten ppm drift over 4096 samples accumulates 0.04 samples of skew and scales the spectrum by $1e-5$; 10 ppm RMS jitter would raise a noise floor around -90 to -110 dB. Neither is learnable next to any realistic channel noise, so the defaults mostly exercise the machinery rather than the impairment.

4. Implementation Findings

4.1 clock_jitter: timing quantization makes the parameter non-functional

The engine's timing resolution is one fine-grid step, $1/5000$ of a sample period, i.e. 200 ppm, and the sub-branch fraction is discarded by truncation (`phase = int(fractional_position)`), so the effective sample time is `floor(position)/L`. Because the nominal positions land exactly on fine-grid integers (start 1, step 5000), any requested Gaussian sigma well below 200 ppm degenerates into a coin flip: negative perturbations floor down one step, positive ones do nothing. Empirically, requested sigma of 1, 5, 10, and 50 ppm all produce bit-identical outputs with the same seed (maximum difference 0.0), and the realized timing error is a two-level process taking values 0 and -200 ppm with roughly equal probability, mean -102.6 ppm, standard deviation 100 ppm, regardless of the requested value. The jitter_ppm knob is therefore non-functional across its entire physically plausible range, the realized process is neither Gaussian nor of the requested variance, and it carries a systematic -100 ppm bias. Only above the grid step does the realized statistic track the request (requested 200 ppm gives std 208 on a four-level lattice; requested 1000 gives 991). The total perturbation the default setting actually adds, measured against the jitter-free output of the same resampler at $f = 0.4$ cycles/sample, is -69 dB, and it is produced by the wrong process. The root cause is that the standard second half of an arbitrary-phase polyphase resampler, interpolating between the two adjacent branch outputs using the sub-phase remainder μ , was omitted; the remainder is simply thrown away.

4.2 clock_drift: core rate behavior is correct

`clock_drift` gets the essential physics right. The realized fine-grid timing slope matches the request to better than 0.01 ppm at 10, 100, and 1000 ppm, and a tone's frequency scales by $(1 + d)$ as expected. (An apparent +1.2 ppm excess in a first tone-based measurement turned out to be the estimator's floor, confirmed at drift 0.) Truncation costs drift far less than jitter, since the cumulative timing ramp is well resolved on the fine grid; it leaves only a fixed sub-grid delay and a deterministic 200 ppm timing staircase, producing phase-modulation spurs on the order of -80 dBc, which is negligible. Per-branch gain normalization (the multiplication by `uprate` in `partition_polyphase` applied to the unscaled prototype) is

correct; measured amplitude ripple as the phase cycles under 1000 ppm drift is $5e-7$ peak to peak.

4.3 Hard bug: a band of drift values returns an empty array

The shared trim logic `data[slice_front:-slice_back]` with `slice_back` equal to 0 evaluates to `data[1:0]`, an empty array, and `slice_back` is 0 exactly when the resampler produces one excess sample. That happens for real parameter values: with $N = 4096$ and this filter (24 taps per branch), every drift from roughly 5,615 to about 5,860 ppm returns a length-zero array. The failing band is centered near $d = (\text{taps_per_phase} - 1)/N$ and has width of about $1e6/N$ ppm, so it moves with signal length and filter length and will eventually be hit by anyone sweeping percent-scale offsets. The fix is `data[slice_front:len(data)-slice_back]` or an explicit guard.

4.4 Alignment: zero impairment is not an identity

The trim splits the excess samples evenly by parity and ignores the filter's group delay, which is 11.51 input samples here (57,550 fine units), while the front padding contributes 23. The net result is that the returned signal leads the input by a constant 0.49 samples regardless of parameters, so drift 0 (or jitter 0) is not an identity: for a tone at $f = 0.05$ the sample-aligned error is -16 dB, entirely due to this deterministic shift, i.e. orders of magnitude larger than the impairment nominally being applied. A constant fractional delay is benign for shift-tolerant classification, but it breaks identity expectations, silently misaligns any sample-indexed annotations, and means the transform's dominant observable effect at default settings is an artifact of its own plumbing. The trim offsets should be computed from the known filter delay, not by even splitting; that would also make the retained edge samples correspond cleanly to the record instead of extrapolating half a sample past the final input sample against zero padding.

4.5 Fidelity of the underlying resampler

The prototype is designed with cutoff and transition width both equal to $1/(2L)$, so in input-frequency units the measured end-to-end response is flat to within 0.03 dB out to 0.35 cycles/sample, -1.9 dB at 0.45, and -6 dB at Nyquist, while image rejection is only -14 dB at 0.55 and -48 dB at 0.65 before reaching the deep stopband near 0.7. For the oversampled signals typical of this kind of dataset that is transparent, but a near-fully-occupied band receives band-edge shaping and image leakage far larger than any ppm-scale clock effect, and this should be documented as a validity condition. Casting the 115k-tap coefficients to float32 also caps the realizable deep stopband somewhat below the 120 dB design target, a secondary point.

4.6 Smaller items

The padding branch (output shorter than input, reachable only at large positive drift) zero-pads both ends, injecting hard discontinuities and shifting content, and transiently promotes to complex128 before the final cast. The `if input_idx > max_input_idx: break` is unreachable because the clip already bounds the position. `ndarray.resize(refcheck=False)` is a fragile growth pattern, though the size estimate makes it effectively dead code. The incremental position accumulation gathers float error harmlessly at these lengths, but `start + k x increment` would be exact. `clock_drift` accepts an rng it never uses; `initial_phase` is exposed on drift but not jitter; the two wrappers duplicate about thirty lines of trim code verbatim; `downrate = copy(uptate)` copies an int pointlessly. The bare `except Exception` around the numba import silently swallows real import errors, so a broken `dsp_numba.py` would silently degrade to the slow path, and nothing enforces bit-compatibility (including RNG call order) between the two paths. Finally, the pure-Python per-output-sample loop plus the Python double loop in `partition_polyphase` make the fallback slow at dataset scale.

5. Verdict

Conceptually: yes, constant sampling-rate offset and white aperture-style jitter, applied as time-base perturbations through a band-limited interpolator, are sound models, subject to the naming problem (offset mislabeled as drift, plus the erroneous “LO drift” docstring), the undocumented decoupling from carrier offset, and the absence of any accumulating or time-varying timing noise, which is what would actually earn the word drift.

Implementationally: `clock_drift` is correct in its core rate behavior but ships a constant half-sample misalignment, an input-dependent parameter band that returns an empty array, and 200 ppm timing granularity. `clock_jitter` is broken for its purpose: below a few hundred ppm the requested magnitude has literally no effect on the output, and the realized process is a biased binary toggle rather than Gaussian jitter.

6. Recommended Fixes

First, keep the sub-phase remainder μ and linearly interpolate between adjacent branch outputs, $y = (1 - \mu) y_p + \mu y_{p+1}$, with carry into the next input index at $p = L - 1$ (or use a Farrow structure). This removes both the 200 ppm quantization and the bias and makes `jitter_ppm` functional at all magnitudes for double the multiply-accumulates; for very small jitter, a first-order alternative $y + \tau y'$ via a differentiator branch is exact to $O(\tau^2)$ and nearly free. Second, replace the parity-based trim with group-delay-aware alignment so that zero impairment is an identity, and fix the negative-zero slice. Third, correct the docstrings and names, state the sign convention (positive `drift_ppm` lowers the effective ADC rate and scales tone frequencies up), state the occupancy assumption of the prototype, and consider adding an accumulating (random-walk or filtered) timing-noise mode so the pair actually covers realistic clock behavior between the two extremes currently modeled.